William Stallings Computer Organization and Architecture 10th Edition

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

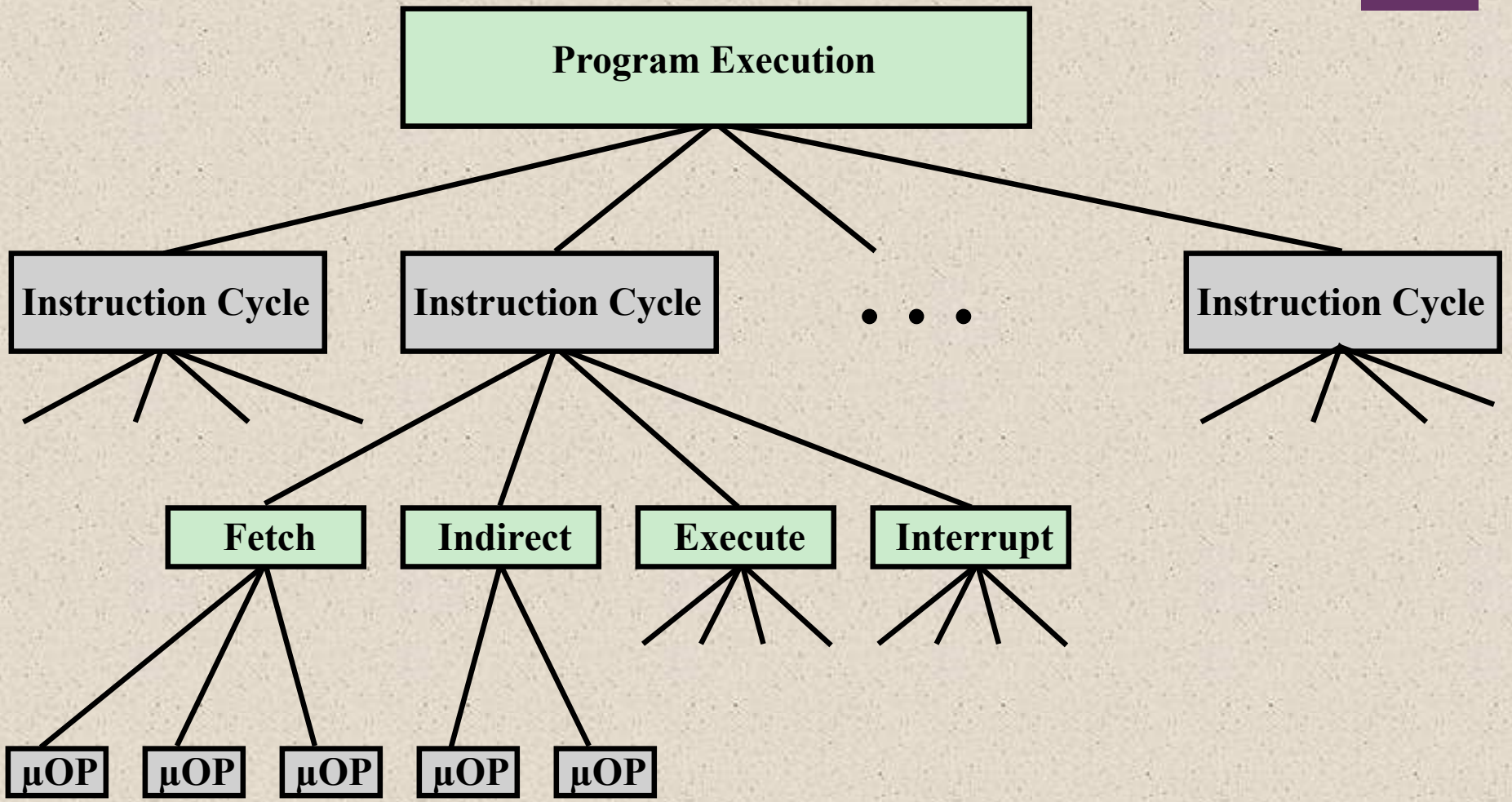


Figure 20.1 Constituent Elements of a Program Execution

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The Fetch Cycle

- Occurs at the beginning of each instruction cycle and causes an instruction to be fetched from memory
- Four registers are involved:
  - Memory Address Register (MAR)
    - Connected to address bus
    - Specifies address for read or write operation
  - Memory Buffer Register (MBR)
    - Connected to data bus
    - Holds data to write or last data read
  - Program Counter (PC)
    - Holds address of next instruction to be fetched
  - Instruction Register (IR)
    - Holds last instruction fetched

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

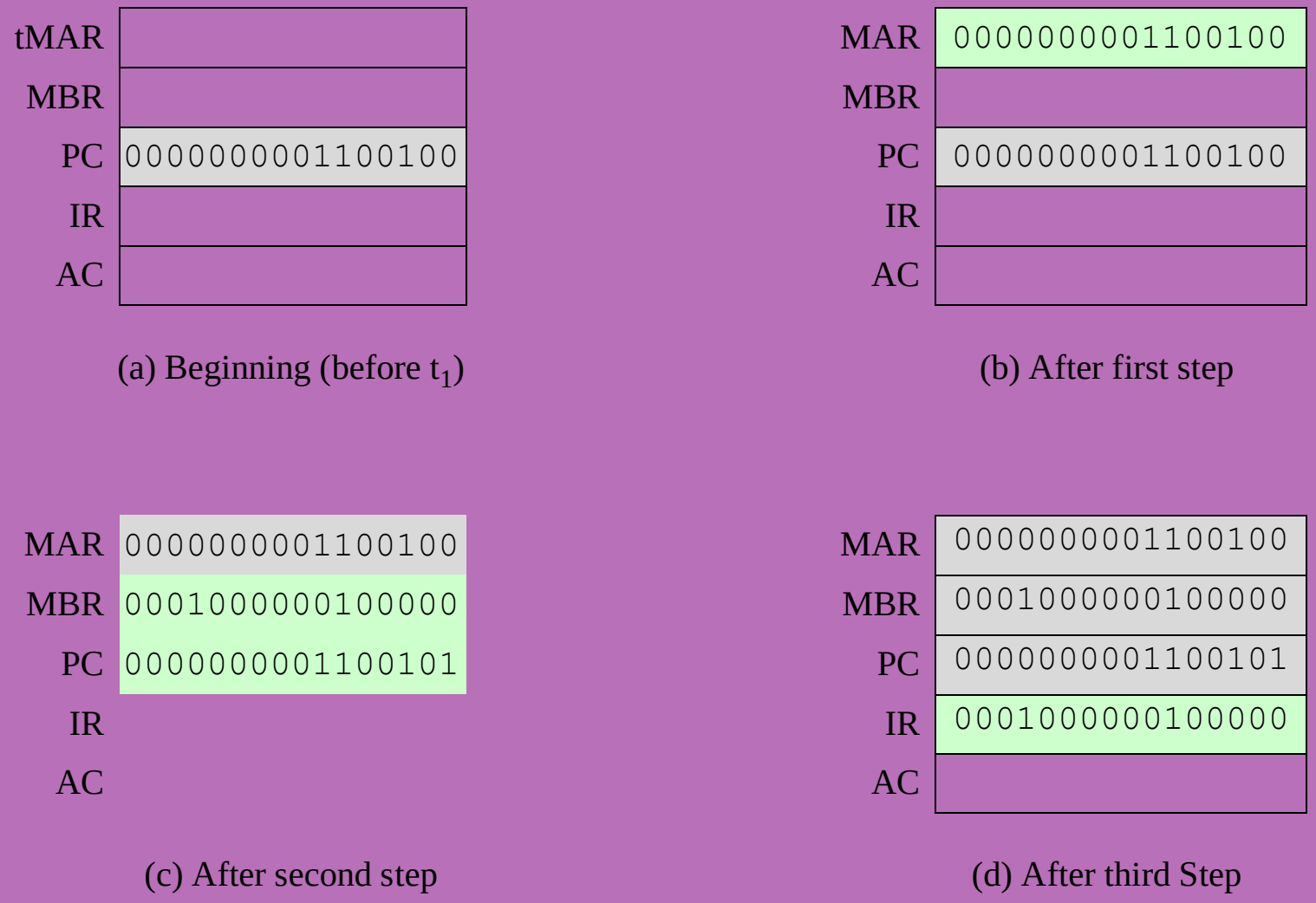


Figure 20.2 Sequence of Events, Fetch Cycle

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

E	E
t1	Move the address from the **Ac cR f e c A** to the **c c dd C de c C** .
t2	Å Place the address in the MAR onto the address bus. Å Control unit issues a READ command on the control bus. Å The instruction from memory at the specified address is placed on the data bus and copied into the Memory Buffer Register (MBR).
t3	Å Transfer the instruction from the MBR to the Instruction Register (IR). Å This frees the MBR for any potential indirect operations.

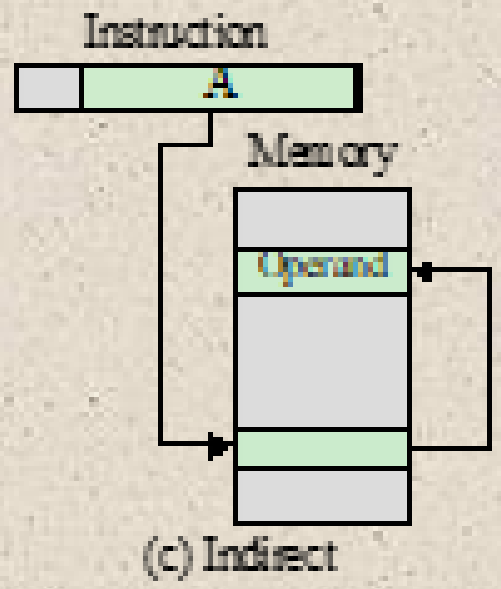
t_1: MAR ← (PC)
t_2: MBR ← Memory
PC ← (PC) + *I*
t_3: IR ← (MBR)

E	c Te T h eY c Te c dd
t1	The address field of the instruction in the Instruction Register (IR) is transferred to the Memory Address Register (MAR).
t2	The address in the MAR is used to fetch the actual operand address from memory, which is stored in the Memory Buffer Register (MBR).
t3	Å Transfer the instruction from the MBR to the Instruction Register (IR). Å This frees the MBR for any potential indirect operations.

t_1: MAR ← (IR(Address))

t_2: MBR ← Memory

t_3: IR(Address) ← (MBR(Address))



E	e ccf ae T
t1	Å The contents of the **Ac cR f e c A** are transferred to the **c f cC de c C**. Å This saves the current instruction address for returning after the interrupt.
t2	Å The **c c dd C de c C** is loaded with the address where the PC's contents will be saved.(A Location in Main Å The PC is then loaded with the address of the start of the **e ccf ae ac T dd c f e** .
t3	Å The PC is then loaded with the address of the **e ccf aeac T dd c f e** . This is where the processor will jump to handle the interrupt.

t_1: MBR ← (PC)
t_2: MAR ← Save_Address
PC ← Routine_Address
t_3: Memory ← (MBR)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

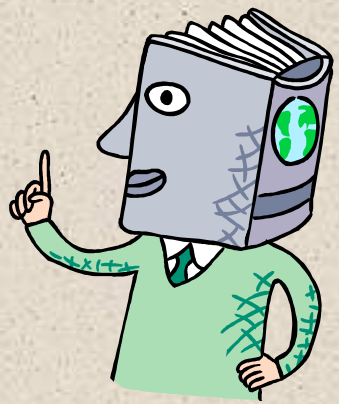
Execute Cycle

- Because of the variety of opcodes, there are a number of different sequences of micro-operations that can occur
- Instruction decoding
  - The control unit examines the opcode and generates a sequence of micro-operations based on the value of the opcode
- A simplified add instruction:
  - ADD R1, X (which adds the contents of the location X to register R1)
    - In the first step the address portion of the IR is loaded into the MAR
    - Then the referenced memory location is read
    - Finally the contents of R1 and MBR are added by the ALU
    - Additional micro-operations may be required to extract the register reference from the IR and perhaps to stage the ALU inputs or outputs in some intermediate registers

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Rules for Micro-Operations Grouping

- Proper sequence must be followed
  - MAR Í (PC) must precede MBR Í (memory)
- Conflicts must be avoided
  - Must not read and write same register at same time
  - MBR Í (memory) and IR Í (MBR) must not be in same cycle
- One of the micro-operations involves an addition
  - To avoid duplication of circuitry, this addition could be performed by the ALU
  - The use of the ALU may involve additional micro-operations, depending on the functionality of the ALU and the organization of the processor



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

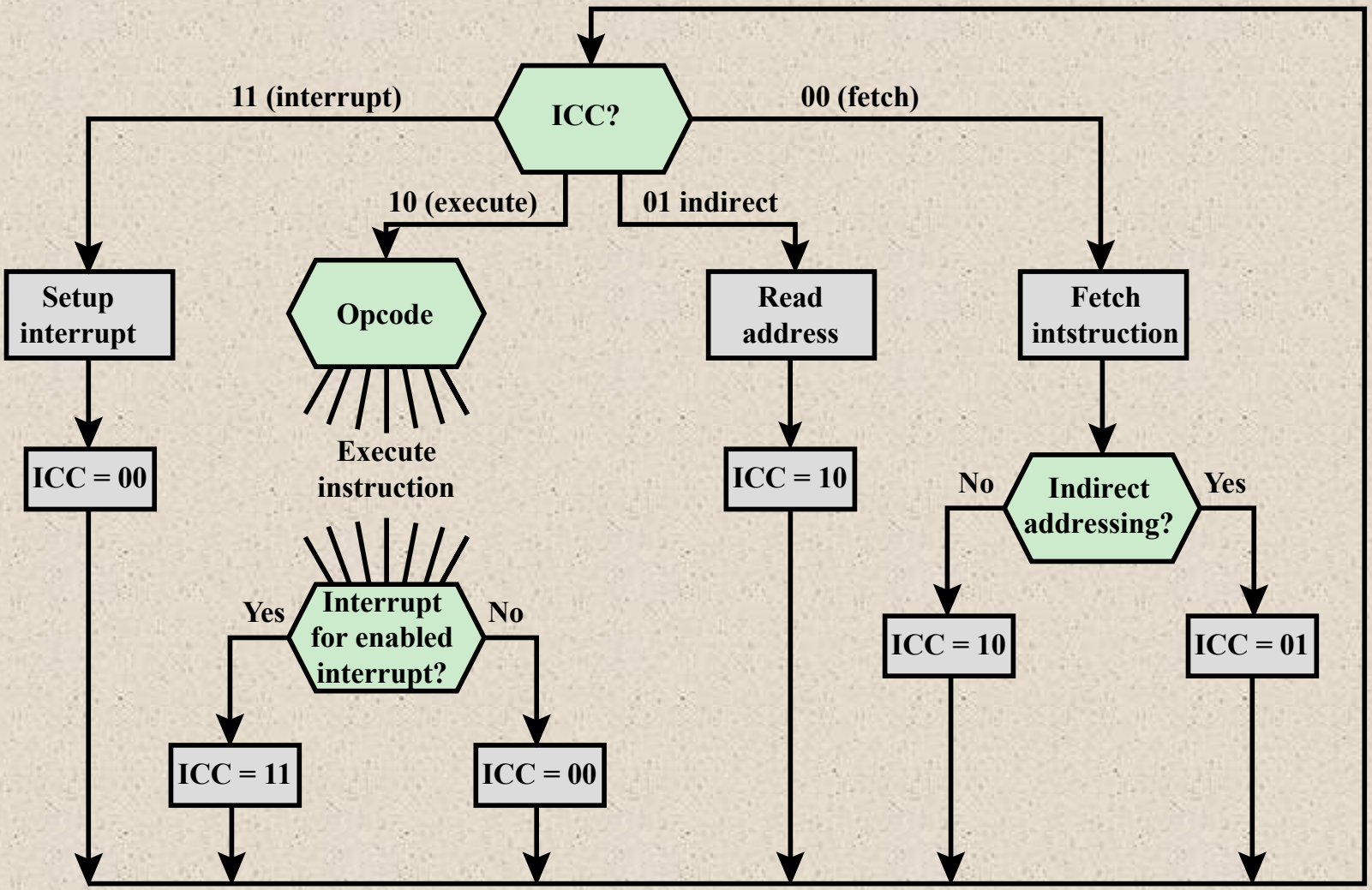


Figure 20.3 Flowchart for Instruction Cycle

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

- **Instruction Cycle Phases**:
- The instruction cycle consists of four main phases:
  - **Fetch**: Retrieving the next instruction from memory.
  - **Indirect**: Resolving indirect addressing (if applicable).
  - **Execute**: Performing the operation specified by the instruction.
  - **Interrupt**: Handling any interrupts that may occur.
- **Instruction Cycle Code (ICC)**:
- A new 2-bit register called the **Instruction Cycle Code (ICC)** is introduced to indicate the current phase of the instruction cycle:
  - 00: Fetch
  - 01: Indirect
  - 10: Execute
  - 11: Interrupt
-

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

- At the end of each cycle, the ICC is updated to reflect the next state.
- **Flow of Cycles**:
  - The indirect cycle is always followed by the execute cycle.
  - The interrupt cycle is always followed by the fetch cycle.
  - For both the fetch and execute cycles, the next phase depends on the current system state.
- **Micro-Operations**:
  - Each phase involves specific sequences of micro-operations that define how the processor interacts with its registers, ALU, and memory.
- **Control Unit Role**:
  - The control unit orchestrates the execution of these micro-operations based on the state of the ICC and the instruction sequence.
  - The control flow is more complex in actual processors but follows a similar structure, emphasizing that processor operation is essentially a sequence of micro-operations

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

HCU vs MCU

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Implementation	Fixed logic circuits	Control memory (firmware)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.